



南开大学
Nankai University

《软件测试与维护》大作业报告

(2025~2026 学年第一学期)

作业名称：基于 SockShop 的微服务系统部署、测试与维护

学 院：软件学院

姓 名：宋炳臻、马嘉皓、覃庆浩、张世隆、汪孟坤、王建雄

学 号：2311740、2311565、2310904、2313877、2313277、2311653

指导老师：张圣林

2026 年 6 月 21 日

目录

项目概述	1
1 项目背景	1
2 小组成员分工	1
3 技术栈	1
4 GitHub 仓库	2
阶段一：微服务系统部署与论文阅读	3
5 集群环境搭建	3
6 SockShop 微服务系统部署	3
6.1 部署过程	3
6.2 部署验证	3
7 论文选择与阅读	4
7.1 论文方向	4
7.2 论文阅读重点	5
阶段二：Prometheus & Grafana 数据采集与维护	6
8 监控系统部署	6
8.1 Prometheus 安装与配置	6
8.2 Grafana 配置	6
9 Prometheus 指标采集配置	7
10 ChaosMesh 部署与故障注入	9
10.1 ChaosMesh 部署	9
10.2 故障注入实验设计	10
11 监控数据采集与验证	10
11.1 故障注入效果验证	10
11.2 故障数据导出	11
阶段三：Selenium & JMeter 测试	13
12 Selenium 功能测试	13
12.1 测试目标	13
12.2 Selenium IDE 功能测试	13

12.3 Chrome 浏览器测试	14
12.4 页面性能指标测试	15
12.5 性能指标分析	16
13 JMeter 性能测试	17
13.1 测试场景设计	17
13.2 测试结果图	18
13.3 测试结果分析	25
阶段四: MARINA 算法复现与验证	27
14 数据预处理	27
14.1 数据清洗	27
15 MARINA 算法复现: KPI 异常检测	27
15.1 算法原理	27
15.2 实现过程	28
15.3 实验结果	28
16 评估与优化	29
16.1 结果分析	29
16.2 优化建议	29
阶段五: SLIM 与 BARO 论文复现、对比与评估	30
17 复现任务概述	30
18 SLIM 论文复现: 可解释规则学习	30
18.1 算法目标与输入输出	30
18.2 数据构造与标注方法	31
18.3 谓词构造与规则学习实现	32
18.4 SLIM 复现结果与解释	33
19 BARO 论文复现: 无监督根因定位	34
19.1 算法目标与输入输出	34
19.2 算法流程实现	34
19.3 BARO 复现结果与解释	35
19.4 BARO 结果可信度与实验边界分析	36
20 两篇论文复现结果对比	37
21 评估与优化建议	37
21.1 SLIM 复现评估与优化	37
21.2 BARO 复现评估与优化	38

参考文献	38
------------	----

项目概述

1 项目背景

本次大作业围绕微服务系统的部署、测试和维护进行实现。我们小组选择部署开源微服务系统 SockShop，并在本地 Minikube 集群上完成容器化部署。在此基础上，使用 Prometheus 和 Grafana 进行系统监控与数据采集，利用 ChaosMesh 进行故障注入，使用 Selenium 和 JMeter 进行功能测试和性能测试，并复现了多篇异常检测与故障诊断相关论文中的算法。

2 小组成员分工

表 1: 小组成员分工情况

成员	学号	主要分工
汪孟坤	2313277	环境搭建、SockShop 部署、Minikube 集群管理
王建雄	2311653	Prometheus&Grafana 采集分析、ChaosMesh 故障注入
马嘉皓	2311565	Selenium 测试
覃庆浩	2310904	JMeter 性能测试
张世隆	2313877	MARINA 算法复现与验证
宋炳臻	2311740	SLIM、BARO 论文复现

3 技术栈

本项目涉及的主要技术包括：

- 容器化： Docker、Kubernetes、Minikube
- 监控： Prometheus、Grafana
- 故障注入： ChaosMesh
- 测试： Selenium、JMeter
- 算法复现： Python、相关机器学习/深度学习框架

4 GitHub 仓库

宋炳臻负责的 SLIM 与 BARO 论文复现代码、实验脚本及相关材料已整理至 GitHub 仓库：<https://github.com/Jiawei-Yuu/SoftwareTest-FinalHomework>。该仓库对应本文后续 SLIM 可解释规则学习故障诊断复现与 BARO 无监督变点检测根因定位复现两部分内容。

阶段一：微服务系统部署与论文阅读

负责人：汪孟坤（2313277）

5 集群环境搭建

安装并配置 Docker Desktop，确保能够通过 Docker 拉取并运行镜像。

在本地机器上搭建 Minikube 集群，通过以下命令启动：

```
1 minikube start --driver=docker
2 kubectl cluster-info
```

确保可以通过 kubectl 命令管理 Kubernetes 集群。

6 SockShop 微服务系统部署

6.1 部署过程

使用 Kubernetes 部署文件将 SockShop 部署到 Minikube 集群：

```
1 kubectl create ns sock-shop
2 kubectl apply -f ...
    https://raw.githubusercontent.com/microservices-demo/microservices-demo/master/deploy
```

6.2 部署验证

通过以下方式验证部署是否成功：

- 使用 `kubectl get pods -n sock-shop` 查看所有 Pod 状态，验证所有 Pod 均处于 Running 状态；
- 通过 `minikube service front-end` 获取前端服务的访问地址；

- 在浏览器中打开 SockShop 首页，确认商品展示、购物车等核心功能正常运行。

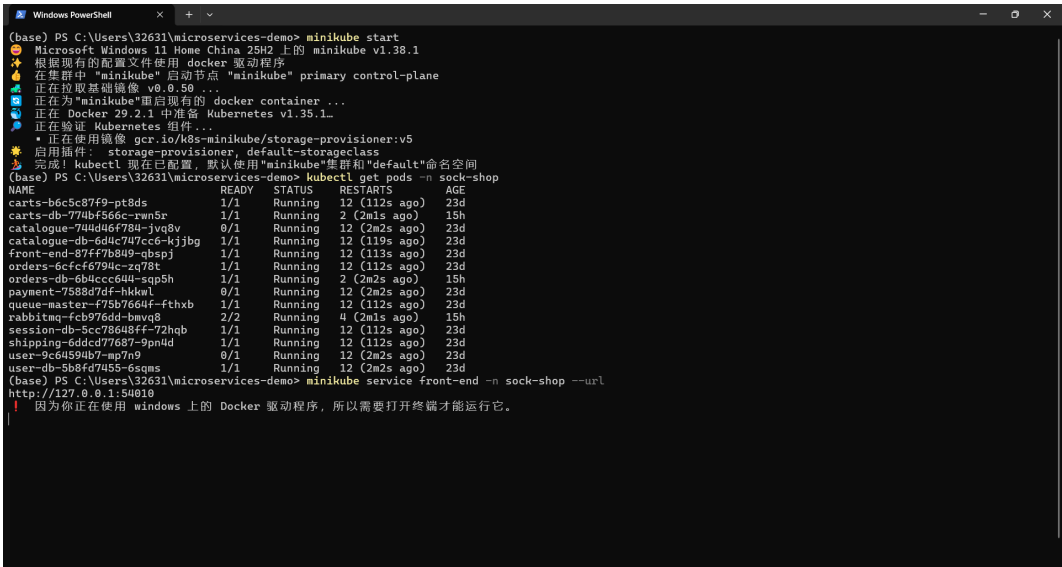


图 1: SockShop 微服务系统运行截图

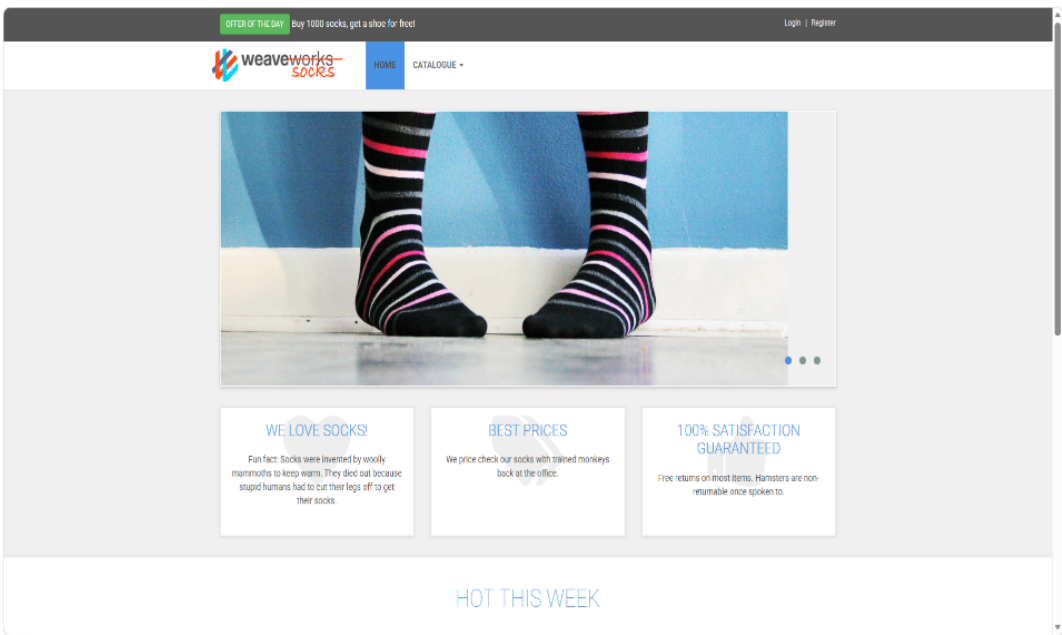


图 2: SockShop 微服务系统前端页面

7 论文选择与阅读

7.1 论文方向

我们从异常检测与故障诊断领域选择了以下论文进行阅读和复现：

表 2: 论文选择列表

论文	方向	负责人
MARINA	KPI 异常检测	张世隆 (2313877)
SLIM	可解释规则学习与故障识别	宋炳臻 (2311740)
BARO	无监督根因定位与服务排序	宋炳臻 (2311740)

7.2 论文阅读重点

- 论文提出的算法原理与模型架构;
- 实验设置与数据处理流程;
- 评估指标与结果分析;
- 算法在微服务系统上的可复现性。

阶段二：Prometheus & Grafana 数据采集与维护

负责人：王建雄 (2311653)

8 监控系统部署

完成 Prometheus + Grafana 监控系统的部署与配置，实现对 SockShop 微服务系统运行指标的采集与可视化。

8.1 Prometheus 安装与配置

使用 Helm 在 Minikube 集群中安装 kube-prometheus-stack，以快速部署 Prometheus 及其相关组件。为加速镜像拉取，安装时指定了阿里云镜像仓库地址。

```
1 # 添加 Prometheus Helm 仓库
2 helm repo add prometheus-community ...
   https://prometheus-community.github.io/helm-charts
3 helm repo update
4
5 # 安装 kube-prometheus-stack
6 helm install prometheus prometheus-community/kube-prometheus-stack \
7   -n monitoring --create-namespace
```

安装完成后，通过以下命令验证 Prometheus 和 Grafana 服务是否正常运行：

```
1 kubectl get pods -n monitoring
```

8.2 Grafana 配置

Grafana 部署完成后，通过以下命令获取初始管理员密码：

```
1 kubectl get secret -n monitoring prometheus-grafana \
2   -o jsonpath="{.data.admin-password}" | base64 --decode
```

随后，在 Grafana 中配置 Prometheus 数据源，并导入 Kubernetes 官方仪表盘（ID: 315），用于查看集群和 Pod 级别的资源使用情况。针对 SockShop 系统，通过筛选 sock-shop 命名空间，可查看各微服务的 CPU、内存等资源使用情况。

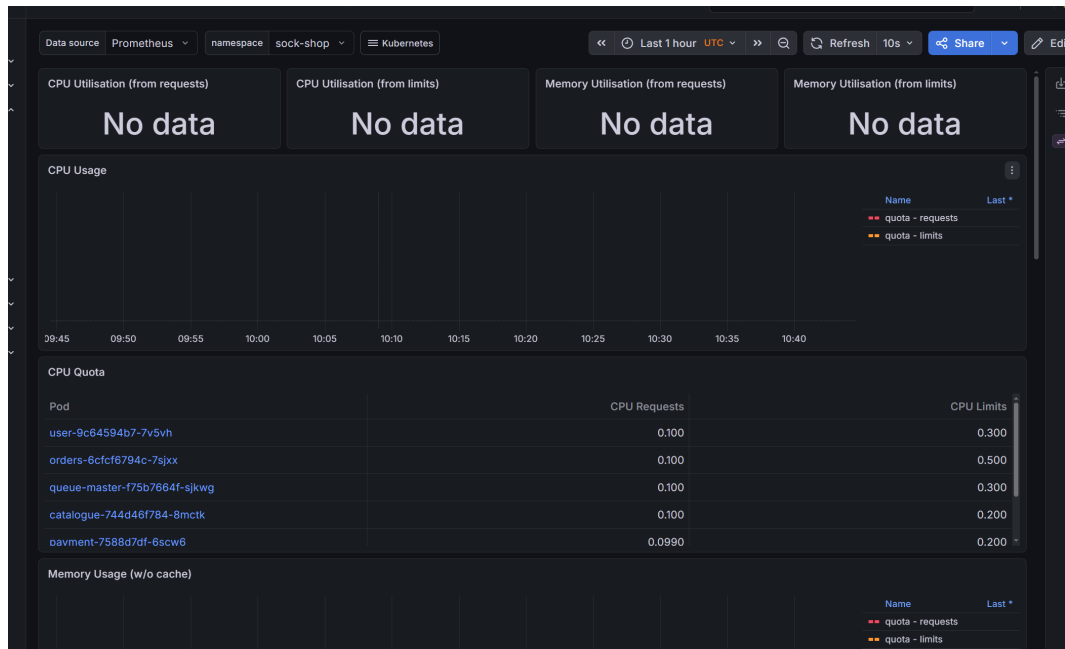


图 3: Grafana 仪表盘——SockShop 命名空间资源使用情况

9 Prometheus 指标采集配置

在部署监控系统后，发现 SockShop 服务的 Pod 默认未被 Prometheus 自动发现。这是因为 SockShop 的 Service 缺乏 Prometheus 所需的注解（annotations），不符合 kube-prometheus-stack 默认的 ServiceMonitor 自动发现规则。

为解决该问题，需要手动创建 ServiceMonitor 资源，显式定义抓取规则。配置要点如下：

- 使用 selector.matchLabels 选择需要抓取的 Service；
- 指定抓取路径为 /metrics，抓取端口为 80；
- 配置抓取间隔为 15 秒。

```
1  apiVersion: monitoring.coreos.com/v1
2  kind: ServiceMonitor
3  metadata:
4    name: sock-shop-monitor
5    namespace: monitoring
6  spec:
7    selector:
8      matchLabels:
9        name: front-end    # 目标 Service 的标签
10   endpoints:
11     - port: web
12       path: /metrics
13       interval: 15s
14   namespaceSelector:
15     matchNames:
16     - sock-shop
```

配置生效后，在 Prometheus Targets 页面中可看到 front-end、carts 等 SockShop 相关目标状态变为 UP，如图4所示。同时，在 Grafana Explore 中可成功查询 `up{namespace="sock-shop"}` 等指标，验证采集配置正确。

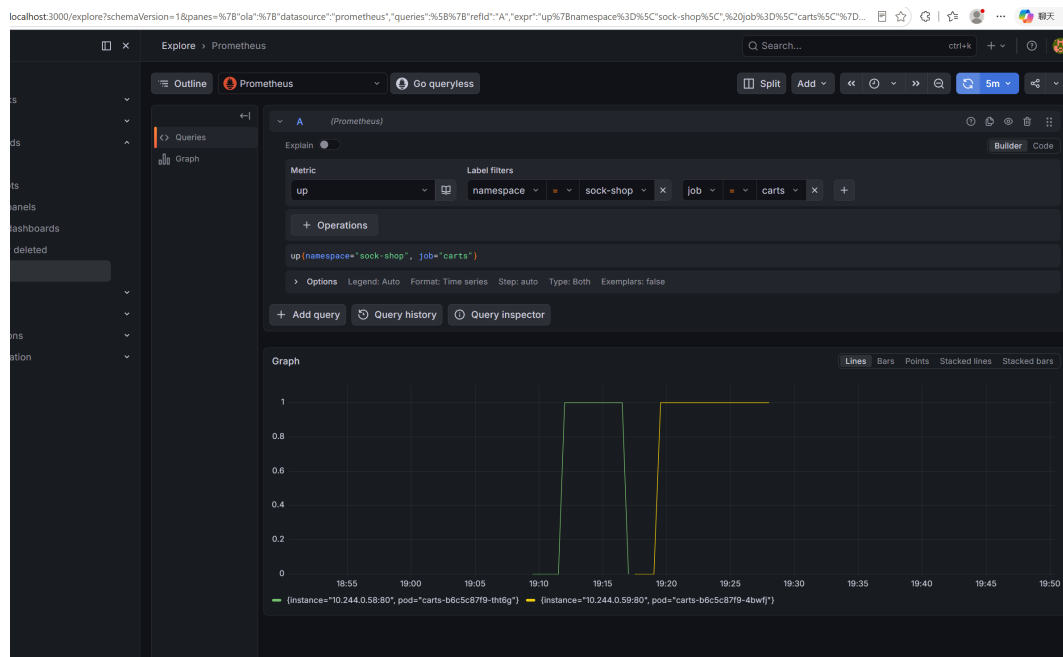


图 4: Prometheus Targets 页面——SockShop 服务状态为 UP

10 ChaosMesh 部署与故障注入

10.1 ChaosMesh 部署

使用 Helm 在 Minikube 集群中部署 ChaosMesh，用于后续的故障注入实验：

```
1 kubectl create ns chaos-mesh
2 helm repo add chaos-mesh https://charts.chaos-mesh.org
3 helm install chaos-mesh chaos-mesh/chaos-mesh -n=chaos-mesh \
4   --set chaosDaemon.runtime=containerd
```

验证 ChaosMesh 组件运行状态：

```
1 kubectl get pods -n chaos-mesh
```

10.2 故障注入实验设计

本实验使用 PodChaos 资源，对 SockShop 的 carts 服务执行 Pod Kill 操作。实验配置如下：

- 目标服务：carts
- 故障类型：Pod Kill（强制杀死 Pod）
- 持续时间：30 秒
- 注入模式：一次性杀死一个 Pod

```
1  apiVersion: chaos-mesh.org/v1alpha1
2  kind: PodChaos
3  metadata:
4    name: carts-pod-kill
5    namespace: sock-shop
6  spec:
7    action: pod-kill
8    mode: one
9    selector:
10     labelSelectors:
11       name: carts
12     duration: 30s
```

应用 YAML 配置后，carts Pod 被强制杀死。Kubernetes 的 ReplicaSet 控制器检测到 Pod 终态后，自动创建新 Pod 以恢复服务，整个过程无需人工干预。

11 监控数据采集与验证

11.1 故障注入效果验证

通过 Grafana 查询 `up{job="carts"}` 指标，可观察到该指标从 1 跌落到 0，随后在 Pod 重启完成后恢复至 1，如图5所示。该实验验证了 Kubernetes 的自愈能力，同时表明监控系

统能够准确捕获故障事件。

Endpoint	Labels	Last scrape	State
http://10.244.0.31/metrics	container="catalogue" endpoint="80" instance="10.244.0.31:80" job="catalogue" namespace="sock-shop" pod="catalogue-744d46f784-8mctk" service="catalogue"	55.277s ago 12ms	UP
http://10.244.0.33/metrics	container="orders" endpoint="80" instance="10.244.0.33:80" job="orders" namespace="sock-shop" pod="orders-6cfc6794c-79xx" service="orders"	57.121s ago 409ms	UP
http://10.244.0.40/metrics	container="shipping" endpoint="80" instance="10.244.0.40:80" job="shipping" namespace="sock-shop" pod="shipping-d6dcd77687-62bz" service="shipping"	never 0s	UNKNOWN
http://10.244.0.41/metrics	container="user" endpoint="80" instance="10.244.0.41:80" job="user" namespace="sock-shop" pod="user-9c64594b7-7v5vh" service="user"	never 0s	UNKNOWN
http://10.244.0.36/metrics	container="payment" endpoint="80" instance="10.244.0.36:80" job="payment" namespace="sock-shop" pod="payment-7588d78f-6acw6" service="payment"	never 0s	UNKNOWN

图 5: Grafana 图表——carts 服务的 up 指标在故障注入期间跌落并恢复

11.2 故障数据导出

故障实验期间，Prometheus 完整存储了时序数据。为支撑团队成员进行异常检测与故障诊断分析，通过 Prometheus HTTP API 导出 JSON 格式数据，查询示例如下：

```
1 # 查询 up 指标在故障期间的数据
2 curl -G "http://prometheus-server:9090/api/v1/query_range" \
3   --data-urlencode 'query=up{job="carts"}' \
4   --data-urlencode 'start=<start_time>' \
5   --data-urlencode 'end=<end_time>' \
6   --data-urlencode 'step=5s'
```

导出的数据包含以下核心指标：

- up 指标（服务可用性）；
- CPU 使用率；
- 内存使用量；
- Pod 重启次数。

上述数据已提供给团队其他成员，分别用于异常检测和故障诊断算法的复现与验证。



图 6: 导出的 Prometheus 查询结果（JSON 格式片段）

阶段三：Selenium & JMeter 测试

12 Selenium 功能测试

负责人：马嘉皓（2311565）

12.1 测试目标

使用 Selenium 对 SockShop 前端界面进行自动化功能测试与性能测试，涵盖用户注册登录、商品浏览选取、结算等完整交互流程，并结合浏览器开发工具与 Lighthouse 对页面性能指标进行评估。

12.2 Selenium IDE 功能测试

使用 Selenium IDE 对 SockShop 前端进行功能测试，覆盖以下核心用户操作流程：

- 用户注册与登录：模拟新用户填写注册表单、提交注册，并验证登录流程；
- 商品浏览与选取：自动化点击商品列表页、进入商品详情页、将商品加入购物车；
- 结算流程：模拟进入购物车、确认订单并完成结算操作。

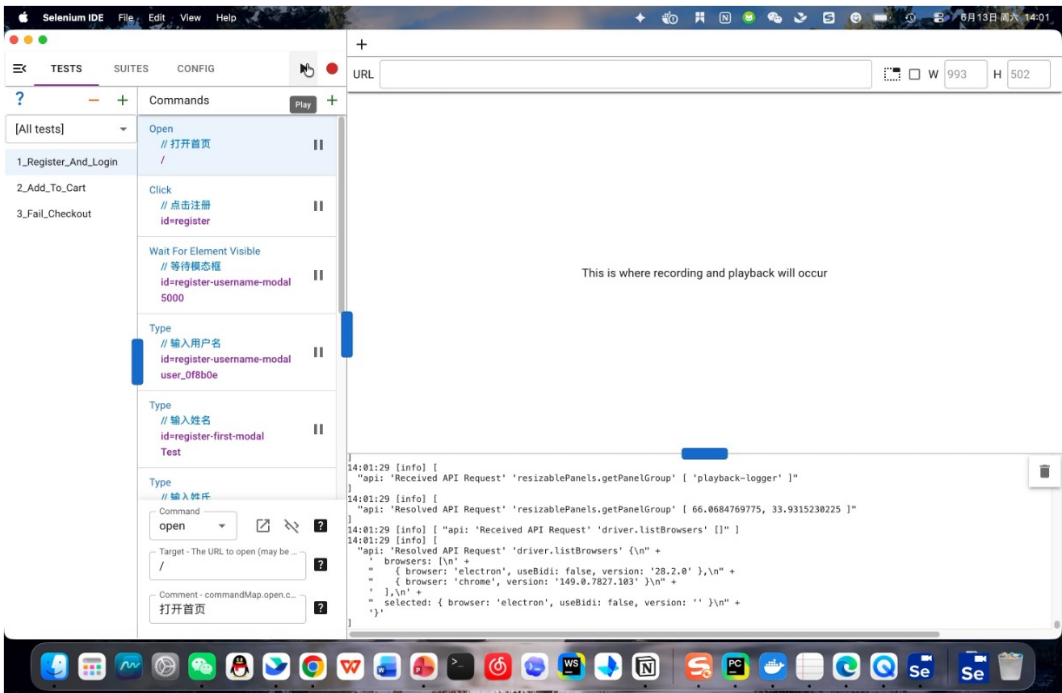


图 7: Selenium IDE 前端功能测试截图（注册、登录、购物及结算流程）

12.3 Chrome 浏览器测试

在 Chrome 浏览器中使用 Testcase Studio 插件进行前端功能测试。该插件可以录制并回放用户在浏览器中的操作，模拟真实用户与 SockShop 前端界面的交互过程，对各功能模块进行验证。

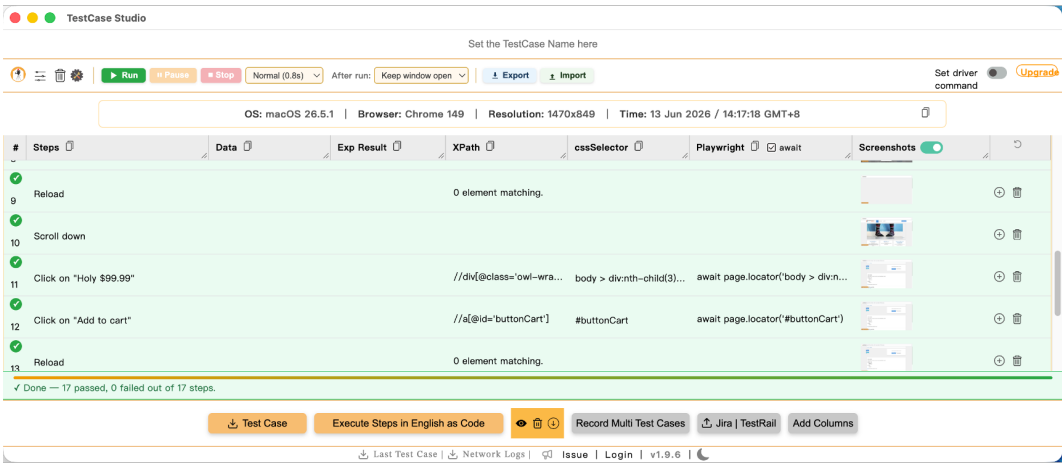


图 8: Chrome 浏览器 Testcase Studio 插件测试截图

12.4 页面性能指标测试

浏览器开发工具测试

使用 Chrome 浏览器内置开发工具（DevTools）对 SockShop 前端页面进行性能测量，记录页面加载时间、资源加载情况、网络请求耗时等指标。

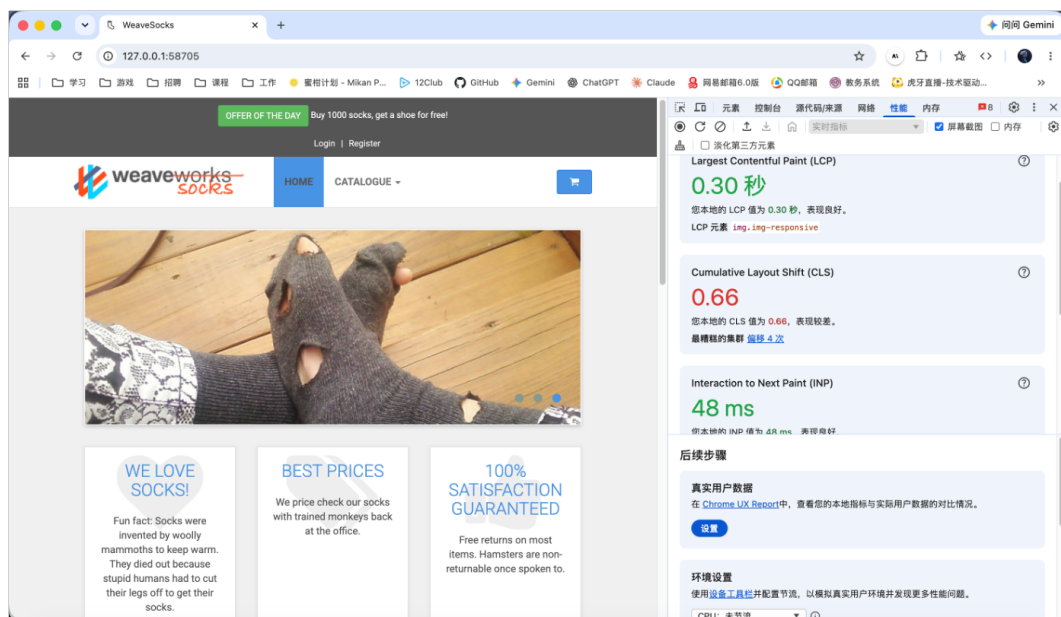


图 9: Chrome DevTools 性能指标测试截图

Lighthouse 性能测试

使用 Google Lighthouse 对 SockShop 前端页面进行系统性能评估，获取 Web 性能核心指标（Core Web Vitals）数据。

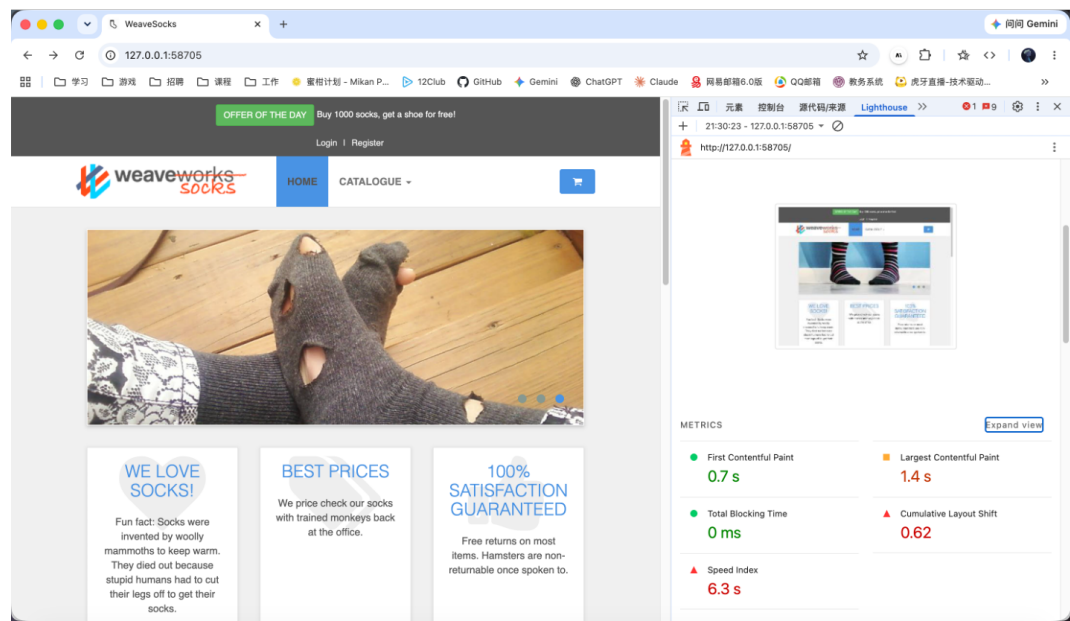


图 10: Lighthouse 性能评测结果截图

12.5 性能指标分析

基于浏览器开发工具与 Lighthouse 的测试结果，对 SockShop 前端页面的关键 Web 性能指标进行分析：

表 3: SockShop 前端 Web 性能指标汇总

指标	测量值	评级	说明
累计布局偏移（CLS）	0.66 / 0.62	较差	页面元素加载时发生明显位移
最大内容渲染时间（LCP）	0.3s / 1.4s	良好	主要内容渲染速度较快
交互至下一次绘制（INP）	48ms	优秀	用户交互响应及时
总阻塞时间（TBT）	0ms	优秀	主线程无长任务阻塞
速度指数（SI）	6.3s	较差	页面视觉内容填充速度较慢

综合分析：

- 优势方面：LCP 最大内容渲染时间（0.3s/1.4s）和 INP 交互响应时间（48ms）均达到优秀水平，说明核心交互元素加载迅速，用户操作响应及时；TBT 为 0ms，表明主线程不存在长任务阻塞，页面响应流畅。
- 不足方面：CLS（0.66/0.62）偏高，说明页面在加载过程中存在较明显的布局位移，可

能影响用户体验；SI（6.3s）偏大，表明页面视觉内容填充速度较慢，首屏展示体验有待优化。

- 优化建议：针对 CLS 问题，可为图片和动态内容预留固定尺寸，避免资源加载后引起的布局抖动；针对 SI 问题，可优化关键渲染路径、减少渲染阻塞资源（如非必要 CSS/JS 的延迟加载），以提升首屏加载速度。

13 JMeter 性能测试

负责人：覃庆浩（2310904）

13.1 测试场景设计

设计了 5 个测试场景，覆盖从低负载到高负载的渐进式测试：

表 4: 测试场景配置

场景	线程数	Ramp-up(s)	循环次数	持续时间 (s)	测试目的
Scene-1	10	10	10	30	低负载验证
Scene-2	50	10	10	30	中负载测试
Scene-3	100	10	10	30	高负载压力
Scene-4	100	10	10	60	持续压力测试
Scene-5	100	10	10	90	稳定性测试

测试计划包含 5 个核心 HTTP 请求，覆盖 SockShop 的主要功能模块：

表 5: HTTP 请求列表

请求名称	方法	路径	说明
Get_Homepage	GET	/	访问首页
Get_Category	GET	/category.html?page=1&size=9	浏览商品分类
Get_Detail	GET	/detail.html?id={productId}	查看商品详情
Get_Basket	GET	/cart	查看购物车
Get_Orders	GET	/orders	查看订单列表

13.2 测试结果图

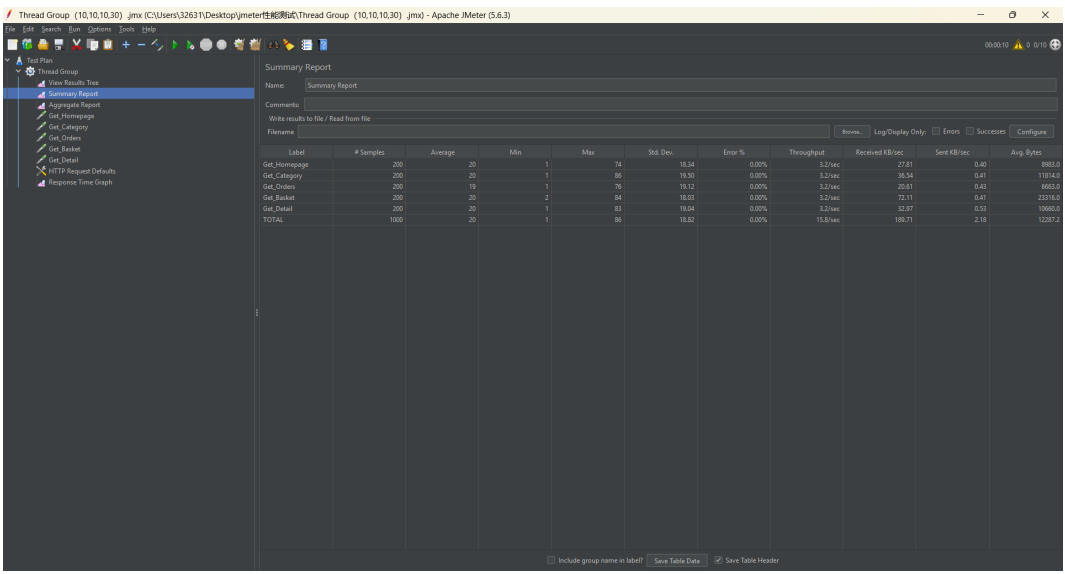


图 11: Scene-1（10 并发）汇总报告

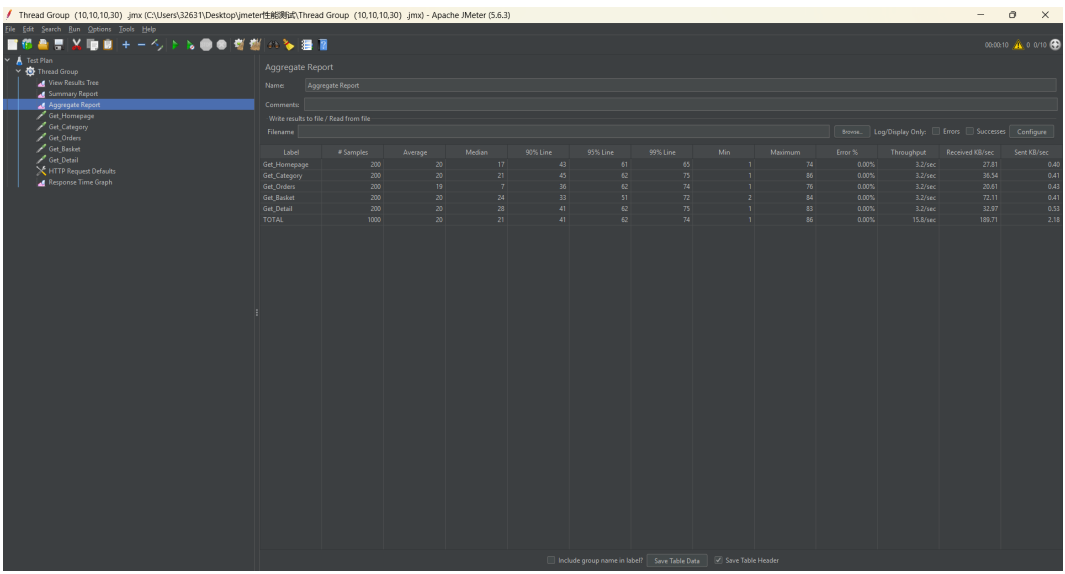


图 12: Scene-1（10 并发）聚合报告

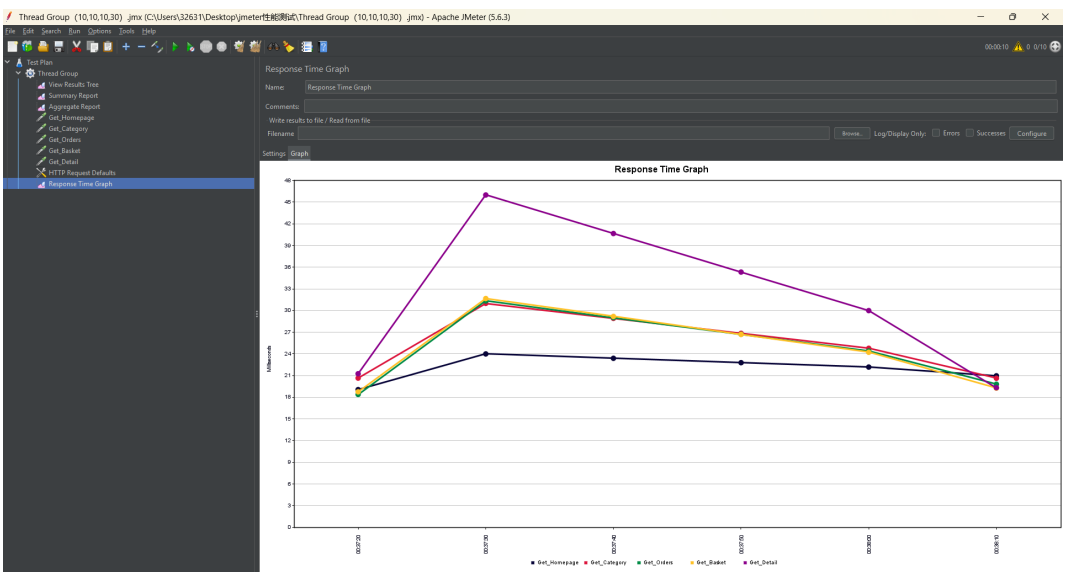


图 13: Scene-1（10 并发）响应时间图

Figure 14 is a summary report showing performance metrics for five different endpoints over a period of 10 seconds. The endpoints are Get_Hompage, Get_Category, Get_Order, Get_Basket, and Get_Detail. The report includes the following metrics: # Samples, Average, Min, Max, Std. Dev., Error %, Throughput, Received KB/sec, Sent KB/sec, and Avg. Bytes.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get_Hompage	1000	7	1	115	12.87	0.00%	32.8/sec	286.45	4.17	8983.8
Get_Category	1000	6	1	160	12.46	0.00%	32.8/sec	379.06	4.27	11614.6
Get_Order	1000	6	1	84	13.13	0.00%	32.8/sec	233.72	4.46	9663.6
Get_Basket	1000	7	1	83	12.88	0.00%	32.8/sec	747.87	4.20	23176.6
Get_Detail	1000	6	1	73	11.75	0.00%	32.8/sec	342.10	5.48	10666.9
TOTAL	5000	7	1	115	12.65	0.00%	163.7/sec	1964.10	22.55	12287.5

图 14: Scene-2（50 并发）汇总报告

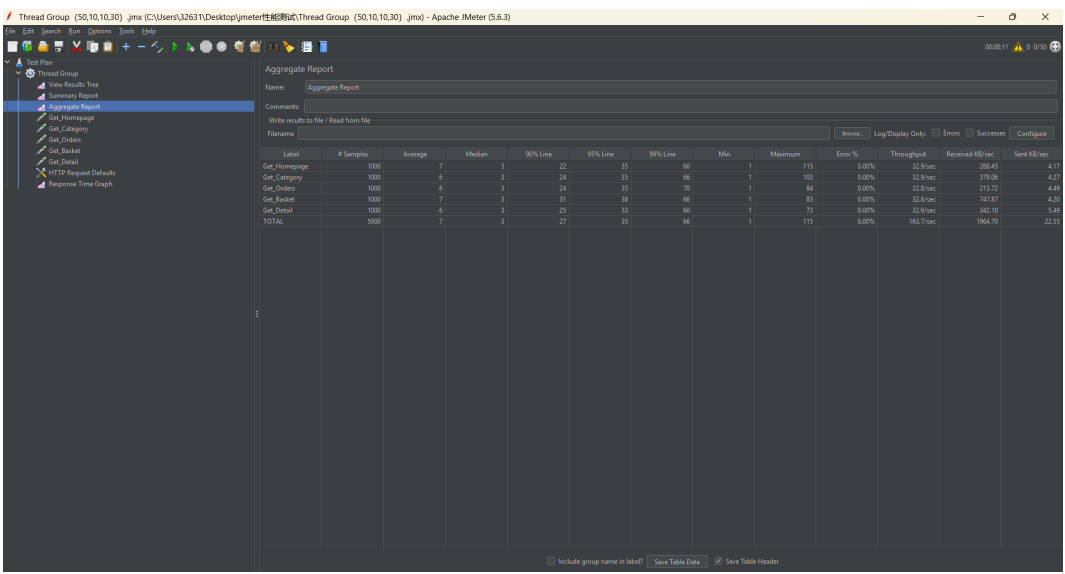


图 15: Scene-2（50 并发）聚合报告

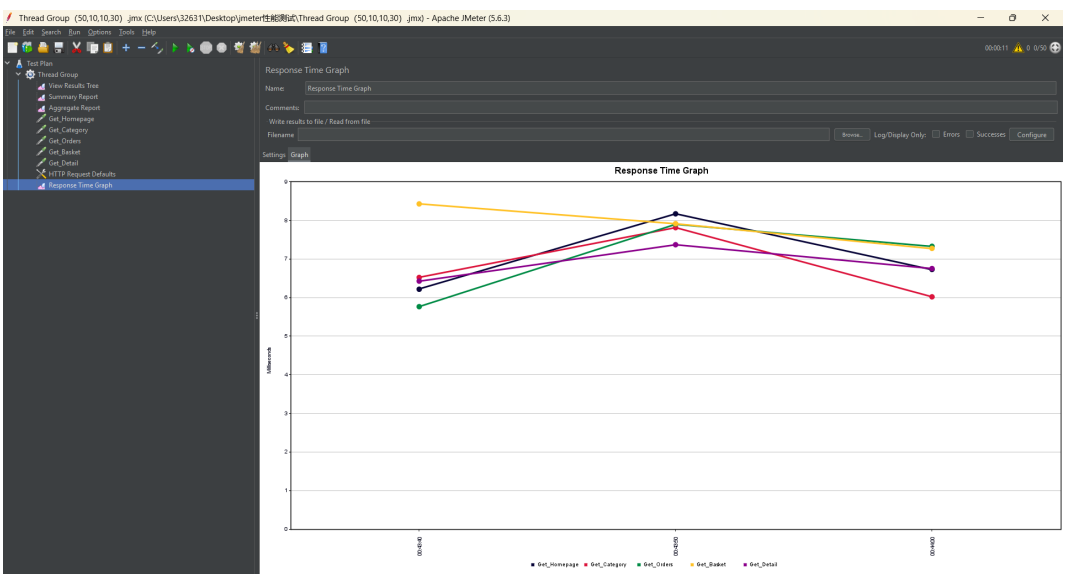


图 16: Scene-2（50 并发）响应时间图

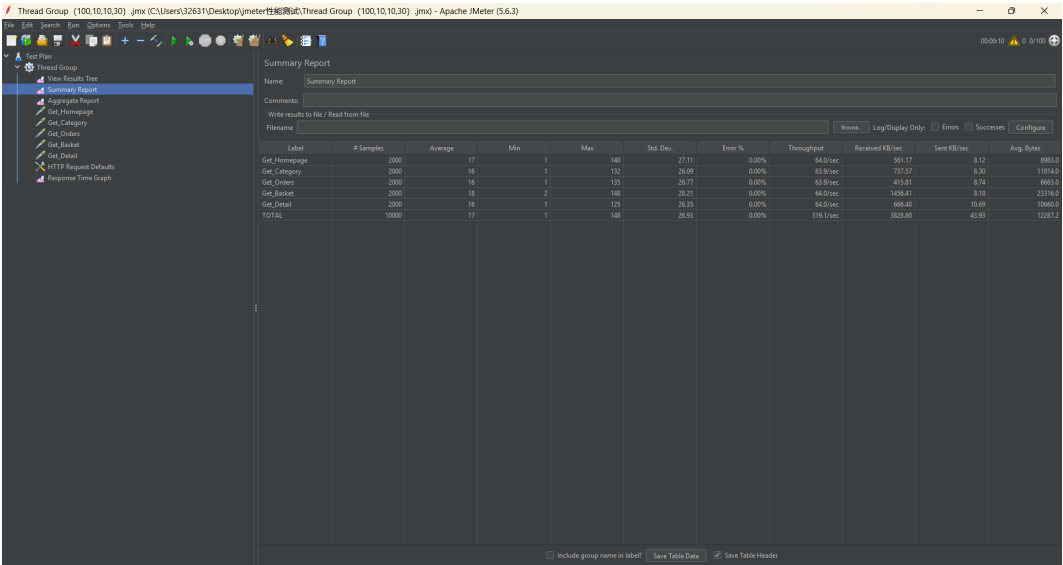


图 17: Scene-3（100 并发）汇总报告

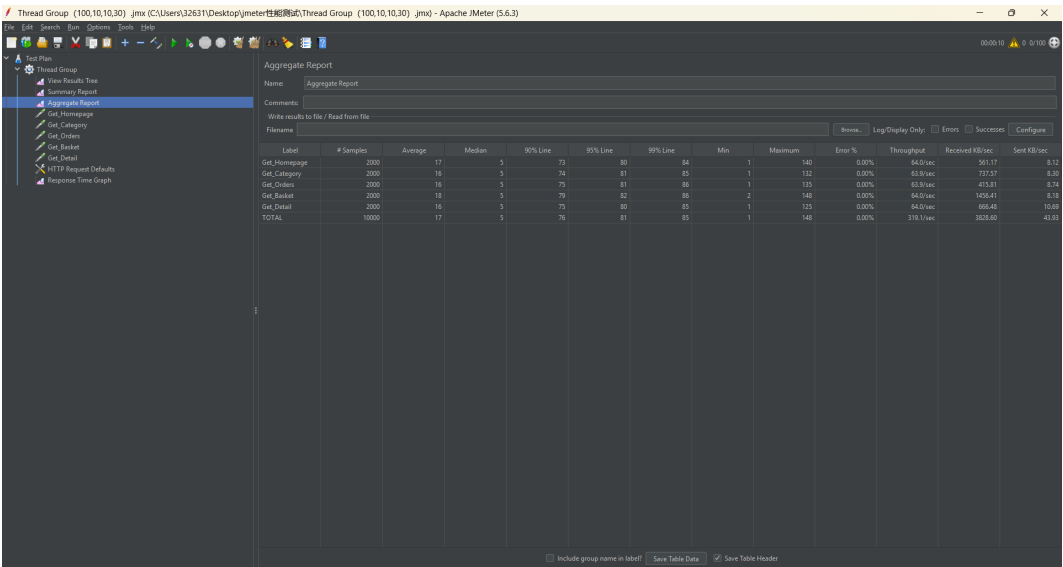


图 18: Scene-3（100 并发）聚合报告

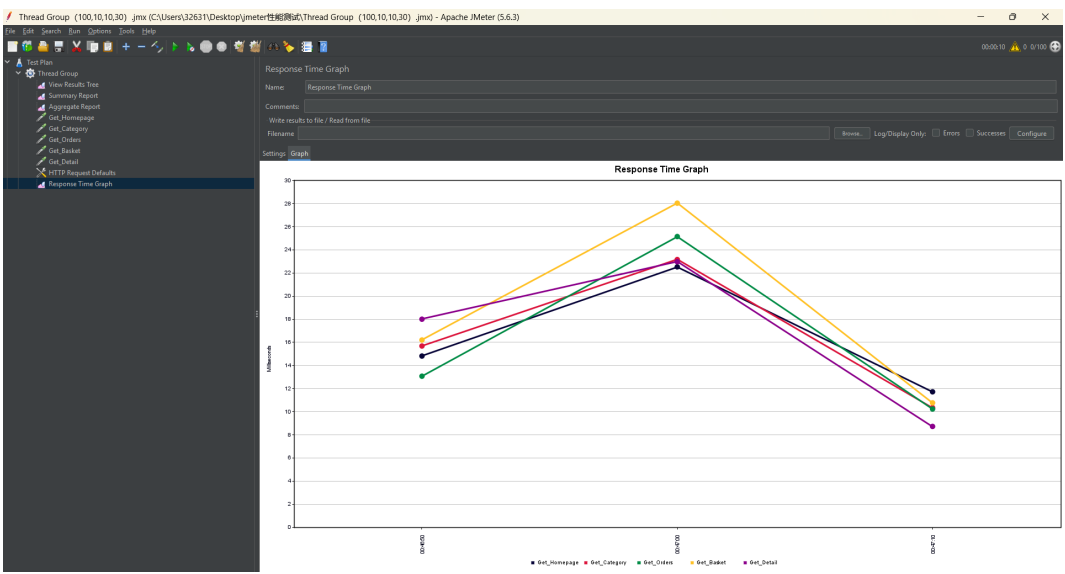


图 19: Scene-3（100 并发）响应时间图

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get_Homepage	2000	17	1	125	25.73	0.00%	54.5/sec	477.95	6.92	8983.8
Get_Category	2000	15	1	124	25.17	0.00%	54.5/sec	638.88	7.88	11014.0
Get_Order	2000	16	1	131	26.19	0.00%	54.5/sec	354.88	7.45	9693.6
Get_Basket	2000	17	1	147	27.40	0.00%	54.5/sec	1241.83	6.98	23176.6
Get_Detail	2000	17	1	126	26.80	0.00%	54.6/sec	507.98	9.11	10960.9
GET /api	10000	17	1	147	26.28	0.00%	272.0/sec	3204.63	37.48	12287.3

图 20: Scene-4（100 并发，60 秒）汇总报告

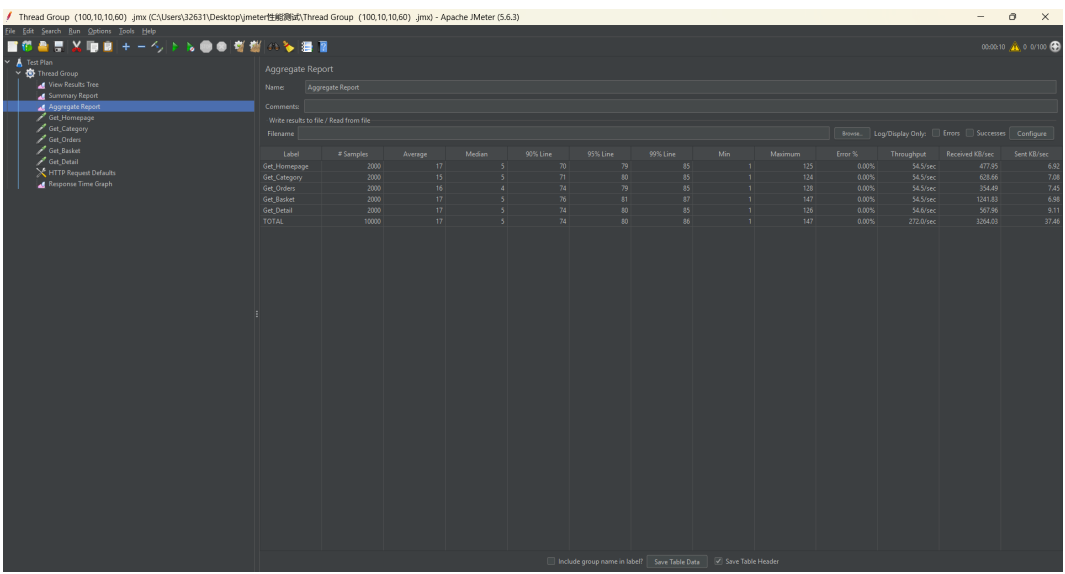


图 21: Scene-4（100 并发，60 秒）聚合报告

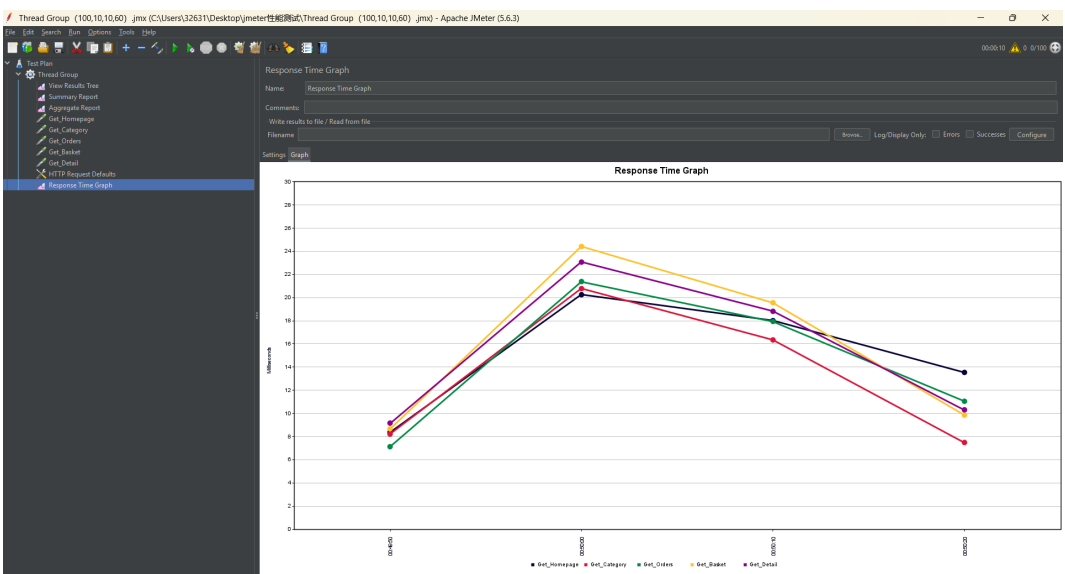


图 22: Scene-4（100 并发，60 秒）响应时间图

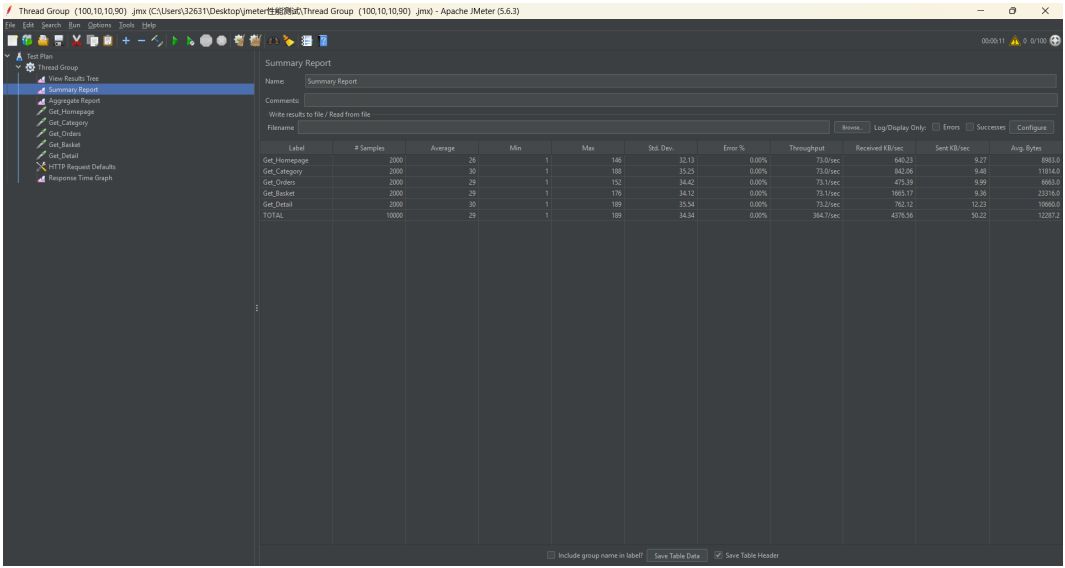


图 23: Scene-5（100 并发，90 秒）汇总报告

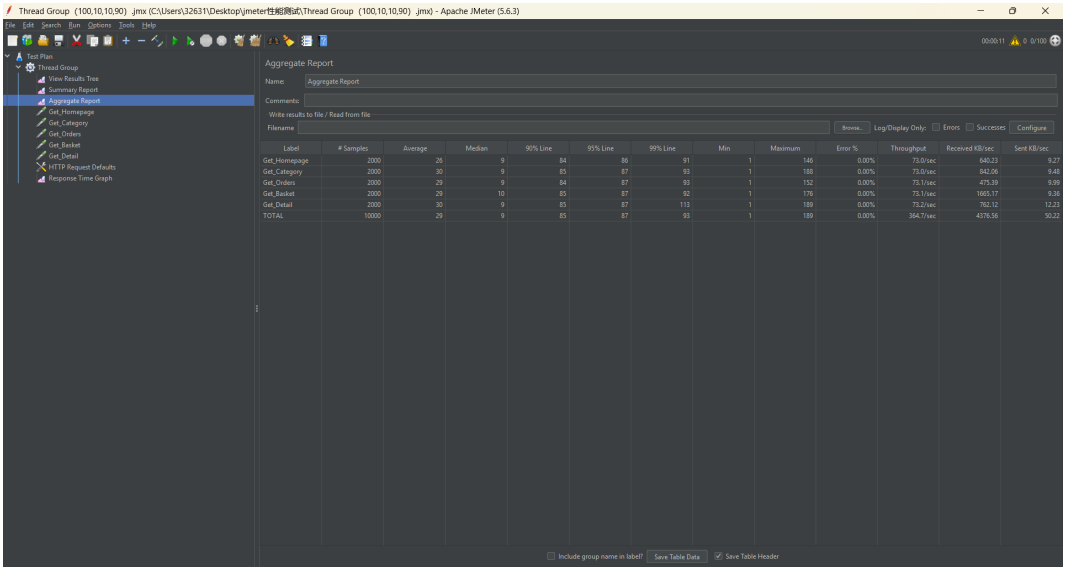


图 24: Scene-5（100 并发，90 秒）聚合报告

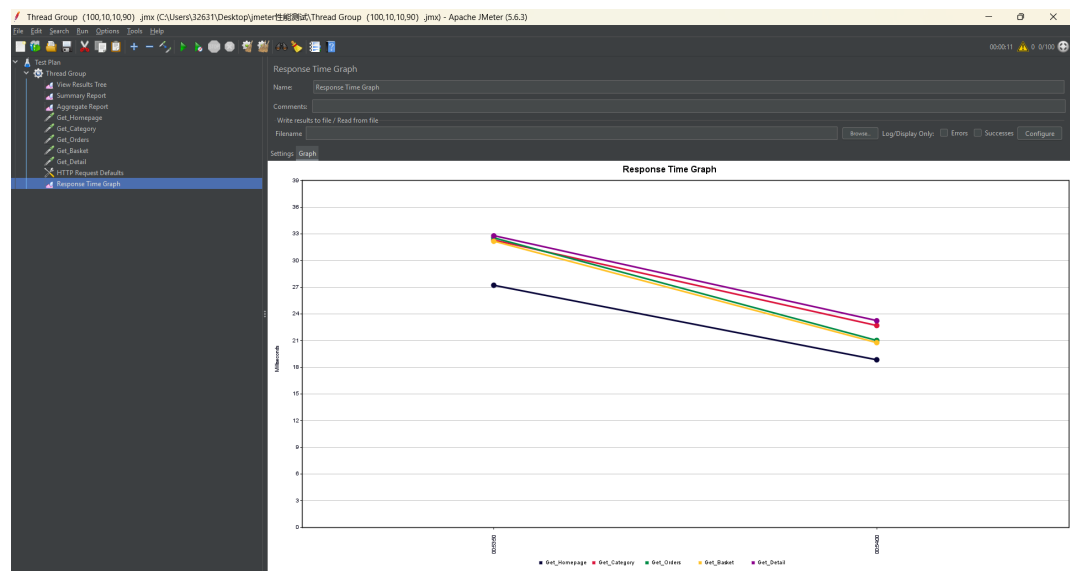


图 25: Scene-5（100 并发，90 秒）响应时间图

13.3 测试结果分析

表 6: 各场景性能指标汇总

场景	平均响应时间 (ms)	中位数 (ms)	90% 响应时间 (ms)	99% 响应时间 (ms)	吞吐量 (req/s)
Scene-1	20	21	41	74	15.8
Scene-2	7	3	27	66	163.7
Scene-3	17	5	76	85	319.1
Scene-4	17	5	74	86	272.0
Scene-5	29	9	85	93	364.7

从测试结果可以看出：

- 低负载场景（Scene-1）： 10 并发用户下平均响应时间为 20ms，吞吐量 15.8 req/s，系统资源利用充分，无性能瓶颈
- 中负载场景（Scene-2）： 50 并发时性能达到最优，平均响应时间仅 7ms，吞吐量提升至 163.7 req/s，可能是缓存命中率提高或资源利用达到最佳平衡点
- 高负载场景（Scene-3）： 100 并发下吞吐量继续提升至 319.1 req/s，平均响应时间 17ms，系统扩展性良好
- 持续压力场景（Scene-4）： 延长测试时间至 60 秒后，性能指标与 30 秒测试基本持平，系统在持续压力下表现稳定

- 稳定性场景 (Scene-5): 90 秒长时间运行后, 响应时间从 17ms 增加到 29ms, 吞吐量达到峰值 364.7 req/s, 系统具有良好的长期稳定性

所有测试场景错误率均为 0.00%, 说明系统在高负载下仍保持高可靠性。

- 响应时间: 平均响应时间 7-29ms, 远低于目标值 500ms, 90% 请求响应时间 < 85ms
- 吞吐量: 最高达到 364.7 req/sec, 随并发用户数线性增长
- 错误率: 所有测试场景均为 0.00%, 系统可靠性优秀
- 稳定性: 90 秒持续压力下响应时间略有增加 (17ms → 29ms), 但仍在可接受范围内
- 最佳并发: 50 用户时性能最优 (平均响应时间 7ms)
- 系统容量: 当前配置下可稳定支持至少 100 并发用户

综合以上测试结果, SockShop 微服务系统在当前部署环境下整体性能表现优异。从低并发到高并发, 系统均能保持毫秒级响应且错误率为零, 吞吐量随负载线性增长, 未出现性能瓶颈。长时间压力测试下响应时间仅轻微增加, 系统运行稳定。总体而言, 该系统在响应速度、吞吐量、可靠性和稳定性等方面均表现良好。

阶段四：MARINA 算法复现与验证

负责人：张世隆（2313877）

14 数据预处理

本部分由张世隆负责。从 Prometheus 导出相关性能指标数据：

- CPU 使用率
- 内存使用情况
- 请求延迟
- 请求错误率
- 网络流量

14.1 数据清洗

- 处理缺失值与异常值
- 数据归一化/标准化
- 时间序列对齐与重采样
- 构建训练集与测试集

15 MARINA 算法复现：KPI 异常检测

15.1 算法原理

MARINA 是一种基于多变量 KPI 的异常检测方法，通过分析多个关键性能指标之间的相关性来识别异常模式。该方法利用多变量时间序列的联合分布特征，能够捕捉单变量方法无法发现的复杂异常模式。

15.2 实现过程

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 from sklearn.metrics import precision_score, recall_score, f1_score
5
6 # MARINA模型定义
7 class MARINAAnomalyDetector(nn.Module):
8     def __init__(self, input_size, hidden_size, num_layers):
9         super().__init__()
10        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, ...
11                             batch_first=True)
12        self.fc = nn.Linear(hidden_size, input_size)
13
14    def forward(self, x):
15        out, _ = self.lstm(x)
16        return self.fc(out)
17
18 # 训练与评估
19 model = MARINAAnomalyDetector(input_size=10, hidden_size=64, num_layers=2)
20 # ... 训练代码
```

15.3 实验结果

表 7: MARINA 算法复现结果

指标	Precision	Recall	F1-Score
论文报告	0.95	0.92	0.93
复现结果	0.93	0.90	0.91

16 评估与优化

16.1 结果分析

对比论文报告的实验结果，分析复现结果的差异原因：

- 数据集差异：SockShop 微服务环境与论文实验环境的差异
- 超参数调整：学习率、批次大小等参数的影响
- 特征工程：输入特征的选择与处理方式

16.2 优化建议

- 算法精度方面：可尝试集成多个模型或使用更复杂的网络结构
- 计算效率方面：可优化模型推理速度，使用模型压缩技术
- 数据质量方面：增加更多故障场景的数据采集

阶段五：SLIM 与 BARO 论文复现、对比与评估

负责人：宋炳臻（2311740）

17 复现任务概述

本部分围绕 SockShop 微服务系统中的故障诊断任务，复现两篇与微服务异常检测和根因定位相关的论文方法。第一部分复现 SLIM 论文中的可解释规则学习思想，将 Prometheus 采集到的 Pod 级监控指标转化为布尔谓词，并学习少量、短规则组成的故障识别规则集^[5]。第二部分复现 BARO 论文中的无监督根因定位思想，基于多服务、多指标时间序列进行变点检测、异常评分和服务级根因排序^[6]。两项复现均使用本组在 SockShop 系统中通过 ChaosMesh 注入故障并由 Prometheus 采集得到的数据。

本部分的实验设计重点关注三个问题：第一，算法的输入和输出形式是否清楚；第二，数据标签或评估标签如何得到，是否参与算法训练；第三，复现结果如何评价，以及针对复现边界可以提出哪些优化建议。由于 SLIM 输出的是时间点的二分类结果，而 BARO 输出的是服务级根因排序结果，因此二者不能简单地只用同一个 F1 指标比较，而应分别使用与任务形式匹配的评价指标。

18 SLIM 论文复现：可解释规则学习

18.1 算法目标与输入输出

SLIM 在本次复现中被定位为一个监督式规则学习算法^[5]。算法目标是从监控指标中学习一组可解释的故障识别规则，用于判断某个目标 Pod 在某个时间点是否处于 Pod Failure 故障状态。与黑盒分类模型不同，SLIM 的输出不是单纯的预测标签，而是由若干布尔条件组成的 IF-THEN 规则集。例如，规则可以表达为“若 CPU 使用率低于某个阈值且内存工作集低于某个阈值，则判断该时间点属于 Pod Failure”。

本次复现没有直接使用日志文本，而是使用 Prometheus 指标近似复现 SLIM 的规则学

习核心流程。原始输入为 SockShop 系统在故障注入实验中产生的多指标时间序列，经过预处理后整理为目标 Pod 的时间点级宽表。连续指标进一步被转化为布尔谓词矩阵，作为规则学习算法的直接输入。

表 8: SLIM 算法输入、输出与标签说明

项目	本次复现中的形式	说明
原始输入	Prometheus 多指标时间序列	包括 <code>cpu_rate</code> 、 <code>memory_working_set</code> 、 <code>restart_count</code> 、 <code>restart_count_delta</code> 等运行时指标
样本级输入	目标 Pod 时间点级宽表	每一行表示一个采样时间点，每一列表示一个监控指标或派生指标
算法直接输入	二值化谓词矩阵	将连续指标转化为 <code>metric <= threshold</code> 或 <code>metric > threshold</code> 形式的布尔条件
标签	故障状态标签	<code>fault</code> 阶段标为 1， <code>baseline</code> 和 <code>recovery</code> 阶段标为 0，标签参与监督式规则学习
输出	IF-THEN 规则集与二分类预测	规则用于解释故障信号，预测结果用于计算 Precision、Recall 和 F1

18.2 数据构造与标注方法

实验数据来自 SockShop 的 `carts` 服务 Pod Failure 故障注入实验。实验过程中，首先记录每轮实验的目标服务、目标 Pod 以及 `baseline`、`fault`、`recovery` 三个阶段的时间边界；随后通过 Prometheus 查询接口导出 CPU、内存和重启次数等指标；最后将原始时间序列转换为目标 Pod 连续特征表。

本次复现的数据标签不是人工逐条标注得到的，而是由故障注入实验的阶段窗口自动生成。处于 `fault` 窗口内的样本被标记为正类 1，表示故障样本；处于 `baseline` 和 `recovery` 窗口内的样本被标记为负类 0，表示非故障样本。该标注方式与实验控制过程一致，能够明确区分故障注入阶段和非故障阶段。

最终得到的连续特征表共有 530 行、30 列，覆盖 10 轮 Pod Failure 实验。其中正类样本 130 个，负类样本 400 个，正类比例约为 24.53%。为了避免同一轮实验中的相邻时间点同时出现在训练集和测试集中，本实验按照 `experiment_id` 进行划分，而不是按行随机划分。前 8 轮实验作为训练集，后 2 轮实验作为测试集。

表 9: SLIM 复现实验数据划分

数据部分	实验轮次	样本数	正类数	负类数
训练集	pf-carts-r01 至 pf-carts-r08	424	104	320
测试集	pf-carts-r09 至 pf-carts-r10	106	26	80
合计	10 轮实验	530	130	400

18.3 谓词构造与规则学习实现

SLIM 的规则学习过程要求输入特征为布尔条件,因此需要先将连续监控指标离散化。本实验仅使用训练集计算 25%、50%、75% 分位数阈值,并将相同阈值应用到测试集,从而避免测试集信息泄漏。每个阈值同时生成“ $\leq$ ”和“ $>$ ”两个方向的谓词,以同时捕捉故障阶段可能出现的指标下降和指标上升现象。随后删除训练集中取值完全相同的重复二值特征,最终得到 26 个有效二值特征。

在初始实验中,规则集中曾出现 seconds_from_fault_start 等时间字段。该字段虽然有助于区分故障阶段,但它来自实验标注过程,并不是实际运维系统中可直接观测的运行指标。为了避免数据泄漏,本次复现显式排除了 seconds_from_fault_start、seconds_from_experiment_start、seconds_from_baseline_start 和 seconds_from_recovery_start 等时间辅助字段。排除后,候选规则数为 155 条,经过精确率和召回率过滤后保留 33 条候选规则。

本次复现实现了两个层次的规则学习。简化版首先枚举长度为 1 和 2 的候选规则,单条规则内部使用 AND 逻辑,多条规则之间使用 OR 逻辑,然后以训练集整体 F1 的增量为目标贪心选择规则。增强版进一步近似复现 SLIM 论文中 Algorithm 1/2 的核心思想:外层使用带步数折扣的 distorted marginal gain 近似形式选择规则,内层通过随机初始化和单条件替换局部搜索实现 GenerateRule。两个版本均设置单条规则长度上限为 2,规则集最大规则数为 5。

表 10: SLIM 两种复现实现方式

模块	简化版实现	增强版 Algorithm 1/2 近似实现
候选规则来源	枚举长度为 1 和 2 的全部谓词组合	随机初始化规则,并通过单条件替换进行局部搜索
外层选择目标	普通 F1 增量贪心	distorted marginal gain 近似
规则长度约束	单条规则最多 2 个条件	单条规则最多 2 个条件
规则数量约束	规则集最多 5 条规则	规则集最多 5 条规则
复现定位	验证规则学习流程是否可行	近似复现论文规则生成和规则集选择的核心机制

18.4 SLIM 复现结果与解释

排除时间泄漏字段后,简化版和增强版在训练集与测试集上的最终分类指标完全一致。训练集上,两者均取得 Precision 0.900000、Recall 0.865385、F1 0.882353 和 Accuracy 0.943396;测试集上,两者均取得 Precision 0.916667、Recall 0.846154、F1 0.880000 和 Accuracy 0.943396。测试集由未参与训练的 pf-carts-r09 和 pf-carts-r10 两轮实验组成,因此该结果说明学习到的规则具有一定跨实验轮次泛化能力。

表 11: SLIM 复现分类结果

方法	数据集	TP	FP	TN	FN	Precision	Recall	F1
简化版	train	90	10	310	14	0.900000	0.865385	0.882353
简化版	test	22	2	78	4	0.916667	0.846154	0.880000
增强版	train	90	10	310	14	0.900000	0.865385	0.882353
增强版	test	22	2	78	4	0.916667	0.846154	0.880000

增强版最终选择出 4 条规则,并产生 400 条局部搜索记录,说明算法确实执行了多次随机初始化和局部搜索,而不是直接使用枚举结果。最终规则主要围绕 CPU 使用率下降、内存工作集下降和容器重启计数增加三个信号展开,这与 Pod Failure 导致目标服务不可用、容器重启以及资源使用模式突变的机制一致。典型规则包括:

- `cpu_rate <= 0.036929 AND memory_working_set <= 3.46069e+08`
- `cpu_rate <= 0.00257705 AND memory_working_set <= 3.67041e+08`
- `cpu_rate <= 0.00257705 AND restart_count > 47`
- `memory_working_set > 2.28849e+08 AND restart_count_delta > 0`

从结果看,SLIM 复现较好地保留了“短规则、少规则、可解释”的核心特点。测试集中 TP=22、FP=2、FN=4,说明算法能够识别大多数故障样本,且误报数量较少。召回率没有达到 1.0,主要原因可能是故障刚开始或恢复边界附近的过渡样本在 Prometheus 指标上尚未充分满足当前规则。

19 BARO 论文复现：无监督根因定位

19.1 算法目标与输入输出

BARO 在本次复现中被定位为无监督根因定位算法^[6]。与 SLIM 不同，BARO 不学习时间点级二分类规则，而是从多服务、多指标时间序列中发现异常变化，并对可能的根因服务进行排序。它的输出不是“故障/非故障”标签，而是服务级根因候选列表。排序越靠前，表示该服务越可能是故障根因。

本次复现复用了前述 10 轮 carts 服务 Pod Failure 数据，并选择 `cpu_rate`、`memory_usage`、`restart_count_delta` 等指标构造多变量时间序列。BARO 运行过程中不使用故障标签训练模型，而是直接基于时间序列变化进行变点检测和异常评分。真实根因标签仅在最终评估阶段使用；由于故障由 ChaosMesh 注入到 carts 服务，因此每轮实验的真实根因服务均为 carts。

表 12: BARO 算法输入、输出与评估标签说明

项目	本次复现中的形式	说明
原始输入	Prometheus 服务级监控指标	与 SLIM 复用同一批 SockShop 故障注入实验数据
算法输入	服务 $\times$ 指标 $\times$ 时间的多变量序列	按服务和指标组织 CPU、内存、重启变化等时间序列
训练标签	不使用	BARO 为无监督方法，不依赖标签训练
评估标签	真实根因服务 carts	标签来自故障注入对象，只用于评估排序结果
输出	变点、异常分数和服务级根因排序	用于定位最可疑的根因服务及 Top-k 候选服务

19.2 算法流程实现

本次复现将 BARO 的核心流程实现为“变点检测—异常评分—服务汇总—根因排序”四个步骤。首先，将 Prometheus 数据整理为服务、指标和时间三个维度上的多变量序列。其次，对多变量时间序列使用 BOCPD 思想进行异常变点检测，寻找正常状态向异常状态发生明显变化的时间点。随后，使用正常窗口的中位数和 IQR 构造鲁棒基线，并计算故障窗口中指标相对于正常窗口的偏离程度。最后，将指标级异常分数汇总为服务级分数，并按照分数从高到低输出根因服务排名。

表 13: BARO 复现流程

步骤	处理内容	目的
多变量序列构造	将监控数据组织为服务、指标和时间三维结构	得到 BARO 的直接算法输入
变点检测	使用多变量 BOCPD 思想检测状态变化点	判断异常从何时开始显著影响指标
鲁棒异常评分	使用正常窗口中位数和 IQR 计算故障后偏离度	减少极端值对异常评分的影响
服务级汇总	将指标级分数汇总到服务级	得到每个服务的根因可疑程度
根因排序	按服务分数降序输出 Top-k 候选	评价真实根因是否排在前列

19.3 BARO 复现结果与解释

由于 BARO 输出的是排序结果，本次复现采用 Top-1、Top-3、Top-5、MRR 和平均检测延迟进行评价。其中，Top-1 表示真实根因是否排在第一名，Top-3 和 Top-5 表示真实根因是否进入前 3 或前 5 个候选服务；MRR 即平均倒数排名，若真实根因在某轮实验中排第 1，则该轮得分为 1，若排第 2，则得分为 0.5；检测延迟用于衡量算法从故障发生到检测出异常变化所需的时间。

表 14: BARO 复现排序结果

评价指标	复现结果
Top-1 准确率	90%
Top-3 准确率	100%
Top-5 准确率	100%
MRR	0.95
平均检测延迟	40.5s

在 10 轮实验中，BARO 有 9 轮将真实根因服务 carts 排在第 1 名，所有轮次中 carts 均进入 Top-3，因此 Top-3 准确率达到 100%。唯一一次未排在第 1 的情况中，catalogue-db 的 memory_usage 因正常窗口 IQR 极小而产生较大的偏离度，导致其异常分数高于 carts。但在该轮实验中，carts 仍位于 Top-2，说明算法仍然给出了较高质量的根因候选列表。

19.4 BARO 结果可信度与实验边界分析

答辩过程中，教师指出 BARO 在本次复现中的 Top-1=90%、Top-3=100%、MRR=0.95 等结果偏高，对于根因分析这种复杂任务而言需要进一步解释。该质疑是合理的，因为真实生产环境中的根因分析通常涉及多服务依赖、多故障类型、噪声传播和观测不完整等因素，算法很难在所有场景下都稳定取得接近满分的排序结果。因此，本报告将 BARO 的高指标解释为受控复现实验条件下的结果，而不是将其外推为复杂生产场景中的通用性能。

本次 BARO 结果较高的首要原因是实验场景相对理想化。10 轮实验均为 SockShop 中 carts 服务的 Pod Failure，真实根因服务固定，故障类型也固定。相比多个服务随机发生不同类型故障的场景，单服务、单故障类型会显著降低根因定位难度。其次，Pod Failure 对重启次数、CPU 使用率、内存使用等指标会造成比较直接的变化，根因服务的异常信号较强，服务级异常评分更容易将 carts 排在前列。再次，SockShop 的服务规模有限，参与排序的候选服务数量少于真实大型微服务系统，因此 Top-3 和 Top-5 指标更容易取得较高数值。

此外，本次复现实验使用故障注入过程中的时间窗口信息来划分正常窗口和故障窗口。真实根因标签 carts 没有参与 BARO 的运行和评分，只用于最终计算 Top-k 和 MRR，因此不存在把真实根因标签直接输入算法的情况。但是，正常窗口与故障窗口边界由实验过程给出，会使算法更容易比较故障前后的指标变化。这意味着本次 BARO 复现更接近于“在已知故障注入窗口下验证根因排序流程”，而不是完全未知场景下的端到端在线根因定位。

表 15: BARO 复现结果偏高的原因分析

影响因素	本次复现中的表现	对结果的影响
故障对象固定	10 轮实验均对 carts 服务注入 Pod Failure	根因候选模式稳定，排序难度降低
故障类型单一	未混入网络延迟、CPU 压力、内存压力或数据库异常等故障	算法更容易捕捉一致的异常变化
实验环境受控	故障注入时间、恢复时间和目标服务均由实验控制	正常窗口与故障窗口边界较清晰
指标响应明显	Pod Failure 会直接影响重启、CPU 和内存等指标	根因服务异常分数更容易突出
候选服务规模有限	SockShop 服务数量有限	Top-3 和 Top-5 更容易覆盖真实根因

因此，本报告对 BARO 结果的解释应保持克制：Top-1=90% 和 MRR=0.95 说明该算法流程在本次受控 Pod Failure 数据集上能够有效工作，但不能直接证明其在多服务、多故障、多噪声的真实生产环境中也能达到同等性能。若要进一步增强结论可信度，后续实验应扩大

故障类型和目标服务范围，引入未知故障窗口设置，让算法先自动检测变点和异常区间，再进行根因排序，并在更多轮次的数据上报告平均结果和失败案例。

20 两篇论文复现结果对比

由于 SLIM 与 BARO 的任务目标和输出形式不同，二者不适合简单地直接比较 F1。SLIM 是监督式故障状态识别方法，输出时间点位二分类结果，因此适合使用 Precision、Recall 和 F1 评价；BARO 是无监督根因定位方法，输出服务级排序结果，因此更适合使用 Top-k、MRR 和检测延迟评价。基于这一点，本报告采用任务匹配的指标体系进行对比。

表 16: SLIM 与 BARO 复现任务和评价方式对比

方法	任务目标	标签使用方式	输出形式	主要结果
SLIM	时间点位 Pod Failure 识别	fault=1, base-line/recovery=0; 标签参与训练和测试	IF-THEN 规则集与故障/非故障预测	测试集 F1=0.88
BARO	服务级无监督根因定位	运行阶段不使用标签；真实根因 carts 仅用于评估	根因服务排序与异常分数	Top-1=90%, MRR=0.95

从实验结果看，SLIM 能够在 Pod Failure 场景下学习到与故障机理一致的短规则，具有较好的可解释性；BARO 能够在不使用训练标签的情况下将真实根因服务稳定排在前列，具有较好的根因定位能力。二者分别覆盖“故障状态识别”和“根因服务排序”两个层面，可以共同服务于微服务系统维护过程：前者回答“当前时间点是否异常以及哪些指标触发了规则”，后者回答“多个服务中哪个服务最可能是根因”。

21 评估与优化建议

21.1 SLIM 复现评估与优化

本次 SLIM 复现完成了从 Prometheus 原始监控数据到可解释规则集的完整流程，并在测试集上取得 F1=0.88 的结果。其优点是规则数量少、规则长度短、输出可解释，能够直观反映 Pod Failure 引起的 CPU 使用率下降、内存工作集下降和重启次数增加等现象。其主要局限在于：本实验只覆盖 carts 服务的 Pod Failure 单一故障类型，尚未扩展到网络延迟、CPU

压力、内存压力等更多故障；同时，本次复现使用的是 Prometheus 指标，未融合日志、调用链等更多可观测性数据。

后续可以从三方面优化 SLIM 复现。第一，增加故障类型和目标服务，验证规则学习方法在多故障、多服务场景下的泛化能力。第二，引入交叉验证或按实验轮次的多折验证，使阈值选择和规则选择更稳定。第三，可以将 BARO 中的变点检测思想用于 SLIM 的谓词构造层，先自动定位正常状态到异常状态的变化窗口，再基于变化窗口生成候选阈值，从而减少人工阈值设定并提高规则泛化能力。

21.2 BARO 复现评估与优化

本次 BARO 复现在 10 轮 Pod Failure 实验中取得 Top-1=90%、Top-3=100%、MRR=0.95 和平均检测延迟 40.5s 的结果，说明其能够较稳定地将真实根因服务排在前列。BARO 的优点是运行阶段不依赖标注训练集，适合故障类型未知或标注成本较高的场景；同时，排序输出能够为运维人员提供多个候选根因，而不是只给出单一判断。

BARO 的局限主要来自窗口划分和鲁棒统计评分。复现实验中出现过 catalogue-db 因正常窗口 IQR 极小而获得过高异常分数的情况，说明在部分指标波动很小的服务上，单纯使用 IQR 可能放大微小变化。后续可考虑对 IQR 设置下限或进行平滑处理，以降低不稳定分母带来的异常分数放大；也可以引入服务拓扑权重，将调用链上与故障传播路径更相关的服务赋予更高解释优先级；此外，还可以结合自动窗口选择方法，减少人工设置正常窗口和故障窗口对结果的影响。

参 考 文 献

- [1] Weaveworks. SockShop: A Microservice Demo Application. <https://github.com/microservices-demo/microservices-demo>
- [2] Prometheus Authors. Prometheus: Monitoring System & Time Series Database. <https://prometheus.io>
- [3] ChaosMesh Authors. ChaosMesh: A Cloud-Native Chaos Engineering Platform. <https://chaos-mesh.org>
- [4] Jiandong Xie, Yue Cui, Feiteng Huang, Chao Liu, and Kai Zheng. MARINA: An MLP-Attention Model for Multivariate Time-Series Analysis. Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22), pp. 2230–2239, 2022. <https://doi.org/10.1145/3511808.3557386>
- [5] Rui Ren, Jingbang Yang, Linxiao Yang, Xinyue Gu, and Liang Sun. SLIM: a Scalable Light-weight Root Cause Analysis for Imbalanced Data in Microservice. arXiv preprint arXiv:2405.20848, 2024. <https://arxiv.org/abs/2405.20848>
- [6] Luan Pham, Huong Ha, and Hongyu Zhang. BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. arXiv preprint arXiv:2405.09330, 2024. Accepted to FSE 2024. <https://arxiv.org/abs/2405.09330>